

Network buffering during data loading

Inferencing example involving sub-processes.

Sub-processes:

- 1) download,
- 2) watch for downloaded files,
- 3) identify a job to be executed

Execution Flow

- Main script
 - While: 1 iteration, no sleep

```
config = SimpleNamespace (...)                                # Reference class to mapping csv files,
                                                            # python config. functions, ftp mount
...
While TRUE                                                    # Formats below
                                                            # are individual
...
mapping = pd.read_csv(config.mapping)                        # .csv format
video_paths = config.videos # convenience alias              # .mp4 format
For row in mapping
    /* Refresh mapping meta information */
    /* Check for bbox/segmentation predictions */ # .csv format
    /*Download vids at particular resolution*/
    For video in row
        helper.tracking_mode(
            trimmed_video_path,
            output_path,
            video_title=f"{vid}_mod.mp4",
            video_fps=video_fps,
            seg_mode=run_seg,
            bbox_mode=run_bbox,
            flag=config.save_annotated_video
        )
    END For
    /*Add bbox/segmentation to results folder*/
    /*Delete .mp4 files*/
    IF exception:
        /*Log email*/
        /*break*/
    END IF
END For
/*Sleep*/
END While
```

Execution Flow

- Tracking_mode(...)
- Ultralytics base Model class
- Ultralytics BasePredictor class

```
For frame in cv2.VideoCapture                                # Uses H.265 encoding!
...
seg_model.track(...)                                       # Function derived from ultralytics Yolo class
bbox_model.track(...)
...
END For
```

```
Model(torch.nn.Module):

    def track(source, stream=True):                        # Treats videocapture as a stream
        ...
        return self.predict(source, stream,...)

    def predict(...):
        return self.predictor(source, stream)              # BasePredictor
```

```
BasePredictor:

    def __call__(source, stream):
        ...
        return self.stream_inference(source, stream,...)

    def stream_inference(...):
        For batch, i in enumerate(dataset)
            results[i] = self.inference(batch)
        End For
        return results

    def inference(batch, ...)
        return self.model(batch)
```

Interlace video download

- Option 1: master script
 - Block on video ID buffer
 - Finishes when outer ID loop finishes
 - No access to async package versions
 - Requires download/predict ratio tuning

```
#!/bin/bash

STATUS_FILE="busy.lock"
ID_BUFFER=() # Used for calls to script_b

start_a() {
    touch "$STATUS_FILE"
}

finish_a() {
    rm -f "$STATUS_FILE"
}

script_b() {
    LOCAL_ID=$1
    predict "$LOCAL_ID"
}

is_a_busy() {
    # Returns 0 (true) if the file exists, or 1 (false) if it doesn't
    [ -f "$STATUS_FILE" ]
}

# --- Master Interlacing Loop ---
for ID in "IDs.txt"; do

    # 1. Start Script A's task
    start_a

    # --- Download video -----
    # First download, then remove lock
    ( download_video "$ID"; ID_BUFFER.push("$ID"); finish_a ) & # Run in background concurrent
    # -----

    # 2. Run Script B repeatedly (polling) WHILE A is busy
    while is_a_busy; do
        for LOCAL_ID in ID_BUFFER
        do
            script_b "$LOCAL_ID"
            ID_BUFFER.pop("$LOCAL_ID")
        done
    done

    # Optional: Ensure the background job has truly exited before proceeding
    wait
done
```

Interlace video download

- Option 2: sync processes
 - Faster than option 1
 - Async video downloading
 - Queue blocked inference scheduling
 - Scheduler: QCG-PilotJob, LocalManager, calculate max. with model mem. limit
 - Writeback: group bbox, dataset mp4
 - Terminate when downloading gives signal

```
import asyncio
from pytube import AsyncYouTube

async def download_video(vid):

    url = f"https://www.youtube.com/watch?v={vid}"
    yt = AsyncYouTube(url, use_oauth=True, allow_oauth_cache=True)

    stream = await yt.get_stream_by_itag(22) # 720p MP4 progressive stream
    if stream is None:
        return

    await asyncio.to_thread( stream.download, output_path="./downloads", filename="{vid}.mp4" )

if __name__ == '__main__':

    IDs = ["YOUR_VIDEO_ID_1", ..., "YOUR_VIDEO_ID_n"]
    download_group = asyncio.gather(*[download_video(id) for id in IDs])

    loop = asyncio.get_event_loop()
    loop.run_until_complete(download_group)
    loop.close()

    with open('./terminate.lock', 'x') as f:
        pass
```

Interlace video download

- Option 2: sync processes
 - Faster than option 1
 - Async video downloading
 - Queue blocked inference scheduling
 - Scheduler: QCG-PilotJob, LocalManager, calculate max. with model mem. limit
 - Writeback: group bbox, dataset mp4
 - Terminate when downloading gives signal

```
from watchdog.events import FileSystemEventHandler
from watchdog.observers import Observer
from predict_script import predict
import pathlib
import queue
import os

class FileCreationHandler(FileSystemEventHandler):
    def __init__(self, file_queue: queue.Queue, watch_dir: str):
        self.file_queue = file_queue
        self.watch_dir = watch_dir

    def on_created(self, event):
        if not event.is_directory:
            filename = os.path.basename(event.src_path)
            self.file_queue.put(filename)

class FileWatcher:
    def __init__(self, directory_to_watch: str):
        self.queue = queue.Queue()
        self.directory = directory_to_watch
        self.event_handler = FileCreationHandler(self.queue, self.directory)
        self.observer = Observer()

        # Ensure the directory exists
        pathlib.Path(self.directory).mkdir(parents=True, exist_ok=True)

    def start(self):
        self.observer.schedule(self.event_handler, self.directory, recursive=False)
        self.observer.start()

    def stop(self):
        self.observer.stop()
        self.observer.join()

    def get(self, timeout=None) -> str:
        return self.queue.get(block=True, timeout=timeout)

def watch(watcher: FileWatcher):
    # Normal scheduling
    while not os.path.exists("/perminate.lock"):
        new_filename = watcher.get(timeout=10)
        predict(new_filename)

    # Flush remaining files from queue
    while True:
        try:
            new_filename = watcher.get(timeout=10)
            predict(new_filename)
        except queue.Empty:
            break

if __name__ == "__main__":
    file_watcher = FileWatcher(SCRATCH_NODE_DIRECTORY)
    file_watcher.start()

    # Start the watcher in the main thread
    watch(file_watcher)

    file_watcher.stop()
```